Тема 6. Работа с JSON-файлами.

хекслет колледж

Цель занятия:

Научить студентов работать со сложными структурами данных в файловой системе. Освоить безопасное преобразование JS-объектов в формат JSON и обратно, а также закрепить асинхронный подход.

хекслет колледж

Учебные вопросы:

1. Формат JSON.
2. Десериализация.
3. Сериализация.
4. Обработка ошибок.

хекслет колледж

1. Формат JSON.

JSON (JavaScript Object Notation) - это текстовый формат обмена данными. Несмотря на то, что он произошел от синтаксиса объектов JavaScript, он является независимым от языка форматом.

хекслет колледж

Основные правила формата:

- Данные хранятся в парах "ключ": значение.
- Ключи всегда пишутся в двойных кавычках.
- Значениями могут быть: строка, число, объект, массив, true, false, null.
- Функции, методы и undefined в JSON запрещены.

хекслет колледж

Почему JSON критически важен для Node.js?

Вот три причины, по которым Node-разработчик работает с ним ежедневно:

- **Конфигурация проектов (package.json).** Весь манифест вашего приложения (название, версия, зависимости, скрипты запуска) хранится в формате JSON. Node.js автоматически читает этот файл при запуске команд NPM.

- **Обмен данными с фронтендом (API)**. Когда вы строите сервер на Node.js, общение с браузером (React, Vue и др.) происходит через JSON. Сервер отправляет JSON-ответ, а фронтенд его легко превращает в JS-объект.

- **NoSQL Базы данных (MongoDB)**. Многие современные базы данных, популярные в стеке Node.js, хранят данные в формате, идентичном JSON (BSON). Это позволяет сохранять объекты из кода напрямую в базу без сложных преобразований.

кекслет колледж

Место JSON в потоке данных

Важно понимать, что JSON - это всегда строка (String). Когда мы работаем с данными в Node.js, они проходят через цикл превращений:

- JS Object (в памяти вашего приложения) - с ним удобно работать.
- JSON String (в процессе передачи или записи) - так удобно хранить или передавать по сети.
- JS Object (у получателя) - снова превращается в объект для работы.

хекслет колледж

Сравнение: JS Объект vs JSON

Характеристика	JS Объект	JSON (строка)
Кавычки у ключей	Необязательны (кроме спецсимволов)	Обязательны (только двойные "")
Кавычки у строк	Любые: ', ", `	Только двойные ""
Функции и методы	Разрешены.	Запрещены. Будут удалены при сохранении
Тип undefined	Разрешен.	Запрещен. Свойство будет удалено
Комментарии	Разрешены (//, /* */)	Запрещены
Типы данных	Любые	Только: String, Number, Boolean, Object, Array, null

кекслет колледж

Пример типичного JSON-файла

```
{
  "projectName": "Pirate Tunes",
  "version": "1.0.0",
  "settings": {
    "port": 3000,
    "debug": true
  },
  "tags": ["music", "downloader", "parser"]
}
```

кекслет колледж

Валидаторы JSON

Валидаторы JSON - это инструменты, которые проверяют текст на соответствие строгим правилам формата.

Поскольку Node.js не прощает ошибок в JSON, использование валидаторов - обязательный этап разработки.

хекслет колледж

Типы валидаторов

Онлайн-сервисы: Самый популярный - JSONLint. Вы вставляете текст, и он подсвечивает строку с ошибкой.

Плагины для редакторов: В VS Code валидация встроена по умолчанию - если вы ошибетесь в .json файле, он сразу подчеркнет место красным.

хекслет колледж

JSONLint

Tools Convert Learn Datasets Extension Mac App

Validate Repair Minify Diff CSV YAML Schema Path JWT Tree All Tools

To format and validate your JSON, just copy + paste it below:

```
{
  "projectName": "Pirate Tunes",
  "version": "1.0.0",
  "settings": {
  "port": 3000,
  "debug": true
  },
  "tags": ["music", "downloader", "parser"]
}
```

Validate Clear Compress Sort Keys Copy

Valid JSON 163 B • 6 keys • depth 2 • 2 objects • 1 arrays

екслет колледж

Как проверить JSON в коде?

```
function isValidJSON(text) {
  try {
    JSON.parse(text);
    return true;
  } catch {
    return false;
  }
}
```

хекслет колледж

Главные "враги" валидного JSON:

Запятая после последнего элемента массива или объекта [1, 2, 3,] - ошибка.

Одинарные кавычки: 'key': 'value' - ошибка (в JSON только двойные).

Комментарии: Любые // или /* */ внутри файла - ошибка.

кекслет колледж

2. Десериализация: Метод JSON.parse().

Этот этап - превращение «мёртвого» текста из файла в «живой» объект, с которым может работать логика JavaScript.
В Node.js это происходит в два этапа: сначала мы получаем данные в виде строки, а затем десериализуем их.

хекслет колледж

Десериализация - это процесс восстановления структуры данных из битового потока или строки.
В нашем случае: превращение текстовой строки формата JSON в объект или массив JavaScript.

хекслет колледж

Базовый синтаксис

Метод JSON.parse(text) принимает один обязательный аргумент - строку, возвращает значение или структуру данных.

```
const jsonString = '{"name": "Ivan", "age": 25}';
const user = JSON.parse(jsonString);

console.log(user.name); // Выведет: Ivan (теперь это объект)
console.log(user.age);  // 25
```

кекслет колледж

Алгоритм работы с файлом (Async/Await)
В реальных задачах мы редко парсим готовую строку. Чаще мы читаем её из файла.

```
const fs = require('fs').promises;
const path = require('path');

async function getSettings() {
    const filePath = path.join(__dirname, 'settings.json');

    // Шаг 1: Читаем файл как строку (обязательно указываем 'utf8')
    const rawData = await fs.readFile(filePath, 'utf8');

    // Шаг 2: Превращаем строку в объект
    const settings = JSON.parse(rawData);

    return settings;
}
```

екслет колледж

Критическая важность try...catch

JSON.parse() - один из самых «опасных» методов.

Если строка содержит хотя бы одну ошибку (лишний пробел в ключе, одинарные кавычки, забытая скобка), метод не просто вернет null, а выбросит исключение, которое остановит выполнение всей программы.

хекслет колледж

Правильный подход:

```
try {
    const data = JSON.parse(rawData);
    // Работаем с данными
} catch (err) {
    if (err instanceof SyntaxError) {
        console.error('Ошибка синтаксиса в JSON файле:', err.message);
    } else {
        console.error('Другая ошибка:', err.message);
    }
}
```

кекслет колледж

Особенности, о которых нужно знать:

Типы данных:

- Если в JSON было число, оно станет Number. Если null - станет null.
- Но если там была дата в кавычках "2026-04-09", она останется строкой. Её нужно превращать в new Date() вручную

Deep Copy (Лайфхак):

- Комбинация JSON.parse(JSON.stringify(obj)) - это классический (хоть и не самый быстрый) способ сделать полную глубокую копию объекта в JS, разорвав все связи с оригиналом.

кекслет колледж

Итог:

- Никогда не используйте JSON.parse() без try...catch.
- Убедитесь, что файл прочитан с кодировкой 'utf8', иначе вы попытаетесь распарсить Buffer (двоичные данные), что приведет к ошибке.

хекслет колледж

3. Сериализация: Метод JSON.stringify().

Сериализация - это перевод структуры данных (объекта или массива) в последовательность символов (строку). В Node.js это основной способ «законсервировать» состояние программы, чтобы после её перезапуска данные не пропали.

хекслет колледж

Базовый синтаксис

Метод JSON.stringify(value, replacer, space) принимает три аргумента:

- value: Объект или массив, который нужно превратить в строку.
- replacer (необязательный): Функция или массив для фильтрации полей (обычно передают null).
- space (необязательный): Отступы для создания «человекочитаемого» вида.

кекслет колледж

Три способа превращения в строку

Компактный (Минифицированный)

Используется для передачи данных по сети, чтобы сэкономить трафик.

```
const user = { id: 1, name: "Ivan" };
const json = JSON.stringify(user);
// Результат: '{"id":1,"name":"Ivan"}'
// (в одну строку, без пробелов)
```

кекслет колледж

Красивый (Pretty Print)

Используется для записи в файлы конфигурации или логи, чтобы их мог прочитать человек.

```
const user = { id: 1, name: "Ivan" };
const json = JSON.stringify(user, null, 2);
/* Результат:
{
  "id": 1,
  "name": "Ivan"
}
*/
```

хекслет колледж

Трансформирующий (С фильтрацией)
Используется, когда вам нужно сохранить в JSON не весь объект, а только определенные поля (например, чтобы не отправить случайно пароль или секретный ключ на фронтенд).

```
const user = {
    id: 1,
    name: "Ivan",
    password: "secret_hash",
    email: "ivan@mail.ru"
};

// Сохраняем только id и name
const json = JSON.stringify(user, ['id', 'name'], 2);
/* Результат:
{
  "id": 1,
  "name": "Ivan"
}
*/
```

кекслет колледж

Особенности и потери при сериализации

Важно помнить, что JSON.stringify - это «фильтр», который пропускает не всё:

- **Функции и методы**: Если у объекта есть метод sayHi(), он будет удален.
- **Undefined**: Поля со значением undefined будут удалены.
- **Даты**: Объекты Date превратятся в строки (ISO-формат).
- **Циклические ссылки**: Если объект ссылается сам на себя, метод выдаст ошибку и программа «упадет».

хекслет колледж

Практический пример: Сохранение данных

```
const fs = require('fs').promises;
const path = require('path');

async function saveDatabase(data) {
    try {
        const filePath = path.join(__dirname, 'db.json');

        // 1. Превращаем объект в "красивую" строку
        const jsonString = JSON.stringify(data, null, 2);

        // 2. Записываем файл (обязательно указываем кодировку 'utf8')
        await fs.writeFile(filePath, jsonString, 'utf8');

        console.log('Данные успешно сохранены!');
    } catch (err) {
        console.error('Ошибка при записи:', err.message);
    }
}
```

екслет колледж

Чтение данных:

```
async function readDatabase(path){
    const jsonString = await fs.readFile(path)
    const data = JSON.parse(jsonString)
    console.log(data)
}
```

хекслет колледж

Еще пример, Полный цикл CRUD для JSON-файла:
У нас есть файл database.json со списком пользователей:

database.json > ...

```
1  {
2    "users": [
3      {
4        "id": 1,
5        "name": "Ivan",
6        "balance": 3000
7      },
8      {
9        "id": 2,
0        "name": "Elena",
1        "balance": 2500
2      }
3    ]
4  }
```

кекслет колледж

Нам нужно обновить баланс конкретному пользователю

```
async function updatePlayerBalance(playerId, amount) {
    const filePath = path.join(__dirname, 'database.json');

    try {
        // ШАГ 1: ЧТЕНИЕ
        const rawData = await fs.readFile(filePath, 'utf8');
        const db = JSON.parse(rawData); // Теперь db — это массив или объект

        // ШАГ 2: ИЗМЕНЕНИЕ (Логика в памяти)
        // Ищем нужного игрока по id
        const player = db.users.find(u => u.id === playerId);

        if (!player) {
            throw new Error('Игрок не найден');
        }

        player.balance += amount;
        player.lastSeen = new Date().toISOString();
```

кекслет колледж

```
        // ШАГ 3: СОХРАНЕНИЕ
        const updatedJson = JSON.stringify(db, null, 2);
        await fs.writeFile(filePath, updatedJson, 'utf8');

        console.log(`Баланс игрока ${playerId} успешно обновлен на ${amount}`);

    } catch (err) {
        console.error('Ошибка в цикле CRUD:', err.message);
    }
}

// Запуск
updatePlayerBalance(1, 500);
```

кекслет колледж

Типы

хекслет колледж

Выводы:

- JSON - это текст, а не объект. Для Node.js данные в файле - это просто набор символов. Чтобы начать с ними работать (складывать числа, пушить в массивы), их нужно десериализовать через JSON.parse().

- Асинхронность - наш стандарт. Использование fs.promises позволяет серверу не «зависать» во время чтения тяжелых файлов. Это критически важно для производительности.

- Безопасность через try...catch. Метод JSON.parse() - один из самых хрупких в JavaScript. Любая ошибка в синтаксисе файла (забытая кавычка или запятая) «уронит» всё приложение, если не обернуть вызов в блок обработки ошибок.

- Полный цикл обновления. Чтобы изменить данные, мы всегда делаем круг: Файл → Строка → Объект → Изменение → Строка → Файл. Напрямую «отредактировать» строчку в середине JSON-файла через Node.js невозможно.

кекслет колледж

Контрольные вопросы

- Какой метод превращает строку в объект, а какой - объект в строку?
- Что вы увидите в консоли, если напишете console.log(getData()), где getData - это async функция?
- Зачем при чтении файла через fs.readFile указывать кодировку 'utf8'?
- Что произойдет со скриптом, если JSON.parse попытается обработать битый (неправильный) JSON без try...catch?
- Можно ли изменить одно поле в JSON-файле на диске, не переписывая весь файл целиком?

хекслет колледж